



Τμήμα Μηχανικών Η/Υ & Πληροφορικής
ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
Πανεπιστήμιο Ιωαννίνων

ΜΥΕ023–ΠΑΡΑΛΛΗΛΑ ΣΥΣΤΗΜΑΤΑ ΚΑΙ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

2ο ΣΕΤ ΑΣΚΗΣΕΩΝ (GPU programming)

ΔΗΜΗΤΡΑ ΧΡΙΣΤΙΝΑ ΓΚΑΡΑΒΕΛΑ ΑΜ:5051
cs05051@uoi.gr

Η δεύτερη εργασία αφορά την εφαρμογή Gaussian blur σε BMP εικόνα (1500×1500 , radius=8), με σειριακή συνάρτηση gaussian_blur_serial() και offload στην GPU μέσω OpenMP.

Όλες οι μετρήσεις έγιναν στον υπολογιστή parallax:

Όνομα υπολογιστή	ex2000@parallax
Επεξεργαστής	Intel(R) Xeon(R) Gold 6130 CPU @ 2.10GHz
Πλήθος πυρήνων	64
Μεταφραστής	clang version 18.1.8

Άσκηση 1

Το πρόβλημα

Η άσκηση ζητά τη συμπλήρωση του προγράμματος **cuinfo.cu** ώστε, χρησιμοποιώντας το CUDA Runtime API, να εντοπίζει όλες τις διαθέσιμες GPU συσκευές NVIDIA στο σύστημα και για κάθε μία να εμφανίζει:

- Το όνομα της συσκευής
- Την έκδοση CUDA compute capability
- Τις εκδόσεις CUDA driver και CUDA runtime
- Το πλήθος των streaming multiprocessors (SMs)
- Το συνολικό μέγεθος global μνήμης
- Το συνολικό μέγεθος constant μνήμης
- Το μέγεθος shared μνήμης ανά block

Ανάλυση κώδικα

Αρχικά, χρησιμοποιείται η `cudaGetDeviceCount(&num_devs)` για να προσδιοριστεί αν υπάρχουν διαθέσιμες GPU. Αν `num_devs == 0`, εκτυπώνεται μήνυμα και τερματίζει η συνάρτηση.

```
cudaGetDeviceCount(&num_devs);
if (num_devs == 0)
{
    printf("No CUDA devices found.\n");
    return;
}
```

Καλούμε τις συναρτήσεις `cudaDriverGetVersion(&driverVer)` και `cudaRuntimeGetVersion(&runtimeVer)` για να αντλήσουμε τις εκδόσεις του driver και του runtime αντίστοιχα. Το CUDA Runtime API επιστρέφει την έκδοση σε έναν ενιαίο ακέραιο, όπου η κύρια (major) έκδοση είναι στον χιλιάδικό ψηφίο και η δευτερεύουσα (minor) στον δεκάδικό, π.χ. το 12040 σημαίνει “12.4.0”. ο ακέραιο αποτέλεσμα (π.χ. 12040) μετατρέπεται σε μορφή `major.minor` διαιρώντας το με 1000 για το κύριο μέρος και παίρνοντας το υπόλοιπο δια 10 για το δεκαδικό (δηλαδή `driverVer/1000` και `(driverVer%1000)/10`).

```
/* version driver & runtime (same for all) */
cudaDriverGetVersion(&driverVer);
cudaRuntimeGetVersion(&runtimeVer);
printf("CUDA Driver Version:  %d.%d\n",
       driverVer / 1000, (driverVer % 1000) / 10);
printf("CUDA Runtime Version: %d.%d\n\n",
       runtimeVer / 1000, (runtimeVer % 1000) / 10);
```

Σε βρόχο `for (i = 0; i < num_devs; ++i)`, καλείται `cudaGetDeviceProperties(&dev_prop, i)` και αντλούνται:

- `dev_prop.name`: Εμφανίζει τον «δείκτη» της συσκευής (το `i`) και το όνομά της (`dev_prop.name`)
- `dev_prop.major`, `dev_prop.minor`: δείχνει το compute capability, δύο αριθμούς (`major`, `minor`) που δηλώνουν την αρχιτεκτονική του GPU (π.χ. 6.1).
- `dev_prop.multiProcessorCount`: πόσα streaming multiprocessors (SMs) έχει η κάρτα
- `dev_prop.totalGlobalMem`: το μέγεθος της συνολικής global μνήμης σε megabytes (`totalGlobalMem` διαιρούμενο με 1024^2).
- `dev_prop.totalConstMem`: το μέγεθος της μνήμης για constant variables σε kilobytes (`totalConstMem` ÷ 1024).
- και `dev_prop.sharedMemPerBlock`: πόση shared μνήμη είναι διαθέσιμη ανά block, σε kilobytes (`sharedMemPerBlock` ÷ 1024).

```
/* Retrieve and pretty-print all the necessary information */
for (int i = 0; i < num_devs; ++i)
{
    cudaGetDeviceProperties(&dev_prop, i);

    printf("Device %d: %s\n", i, dev_prop.name);
    printf("  Compute capability:  %d.%d\n",
           dev_prop.major, dev_prop.minor);
    printf("  Multiprocessors (SMs):  %d\n",
           dev_prop.multiProcessorCount);
    printf("  Global memory:             %.2f MB\n",
           dev_prop.totalGlobalMem / (1024.0 * 1024));
    printf("  Constant memory:           %.2f KB\n",
           dev_prop.totalConstMem / 1024.0);
    printf("  Shared memory per block:    %.2f KB\n",
           dev_prop.sharedMemPerBlock / 1024.0);

    printf("\n");
}
```

Εύρεση συνολικού πλήθους των cores μιας NVIDIA GPU

Αν και το CUDA Runtime API δεν δίνει απευθείας το συνολικό πλήθος πυρήνων (cores) μιας NVIDIA GPU, μπορούμε να το υπολογίσουμε με τα παρακάτω βήματα. Αρχικά, καλούμε τη συνάρτηση `cudaGetDeviceProperties(&prop, device_id);` και από τη δομή `prop` διαβάζουμε:

- `prop.major` και `prop.minor` (την έκδοση compute capability, π.χ. 6.1)
- `prop.multiProcessorCount` (τον αριθμό streaming multiprocessors, π.χ. 30 SMs)

Από τον επίσημο “CUDA C Programming Guide” (Appendix G) παίρνουμε πόσους CUDA cores αντιστοιχούν σε κάθε SM για την κάθε compute capability. Μετατρέπουμε την compute capability σε έναν ακέραιο κωδικό (0x61 για 6.1) και ψάχνουμε σε έναν μικρό πίνακα για να βρούμε τα cores/SM. Στην δικιά μας περίπτωση είναι 128 cores/SM.

Αφού έχουμε τον αριθμό cores/SM (128) και τον αριθμό SMs (30), πολλαπλασιάζουμε:
 $\text{totalCUDAcores} = \text{coresPerSM} \times \text{multiProcessorCount} = 128 \times 30 = 3.840$

Έτσι υπολογίζουμε ότι η GPU διαθέτει συνολικά 3.840 CUDA cores, αξιοποιώντας μόνο τις πληροφορίες του `cudaDeviceProp` και τον πίνακα cores/SM.

Παρακάτω φαίνονται τα αποτελέσματα του κώδικα:

```
[ex2000@parallax ~]$ ./cuinfo
CUDA Driver Version: 12.4
CUDA Runtime Version: 12.4

Device 0: Tesla P40
  Compute capability: 6.1
  Multiprocessors (SMs): 30
  Global memory: 22905.94 MB
  Constant memory: 64.00 KB
  Shared memory per block: 48.00 KB
```

Άσκηση 2

Το πρόβλημα

Στην παρούσα άσκηση επεκτείνουμε την υλοποίηση της Gaussian θόλωσης εικόνας που είδαμε στην Εργασία #1, μεταφέροντας τον υπολογισμό από τη CPU στη GPU μέσω OpenMP offloading. Αντί για τη σειριακή συνάρτηση `gaussian_blur_serial()`, η οποία διασχίζει pixel-pixel την είσοδο `imgin` και παράγει την θολωμένη έξοδο `imgout` με δοθέντα ακτίνα `radius`, συμπληρώσαμε τη συνάρτηση `gaussian_blur_omp_device()` ώστε όλη η

επεξεργασία να εκτελείται στην επιταχυντική κάρτα CUDA. Η συσκευή διαθέτει 30 Streaming Multiprocessors (SMs) των 128 threads η κάθε μία, επομένως θα πειραματιστούμε με διάφορους συνδυασμούς ομάδων (teams) και νημάτων ανά ομάδα (threads per team), με τους εξής περιορισμούς:

1. Κάθε ομάδα threads πρέπει να έχει μέγεθος πολλαπλάσιο του warp (32 threads).
2. Ο συνολικός αριθμός ομάδων πρέπει να καλύπτει πλήρως τους $30 \times 128 = 3840$ logical CUDA threads.

Σκοπός είναι να μετρήσουμε και να συγκρίνουμε τους χρόνους εκτέλεσης για διαφορετικές παραμετροποιήσεις, αξιοποιώντας κατάλληλα τις οδηγίες target, teams, distribute και parallel for collapse(2) ώστε να μεγιστοποιηθεί η κατανομή φόρτου και να ελαχιστοποιηθεί το overhead συγχρονισμών.

Μέθοδος παραλληλοποίησης

Δοκίμασα τις παρακάτω υλοποιήσεις χρησιμοποιώντας num_teams(30) και num_threads(128), προκειμένου να συγκρίνω την απόδοσή τους και να βρω ποιο είναι πιο αποδοτικό.

Η χρήση του #pragma omp target έξω από το teams/distribute εξασφαλίζει ότι η μεταφορά όλων των δεδομένων στη GPU γίνεται μόνο μία φορά πριν από την εκτέλεση του kernel, αποφεύγοντας το επιπλέον overhead από πολλαπλά target regions. Στη συνέχεια, το teams distribute parallel for collapse(2) αναθέτει κάθε γραμμή i σε διαφορετικό Streaming Multiprocessor (SM), ενώ μέσα σε κάθε SM τα νήματα εκτελούν παράλληλα “collapsed” ζευγάρια (i,j). Με τον τρόπο αυτό, κάθε νήμα λαμβάνει σχετικά μεγάλο κομμάτι εργασίας (ελαχιστοποιείται το synchronization) και επιτυγχάνεται ομοιόμορφη κατανομή φόρτου (load balance) σε όλα τα SMs. Η ξεκάθαρη διαχώριση των pragmas (teams + distribute parallel for) μειώνει την πολυπλοκότητα που καλείται να διαχειριστεί ο OpenMP runtime σε σχέση με παραλλαγές που συγκεντρώνουν πολλά clauses σε μία γραμμή ή κάνουν nested target εντολές.

```
289 #pragma omp target map(to: inR[0:npix], inG[0:npix], inB[0:npix], \
290                       radius, width, height, sigma)\
291                       map(from: outR[0:npix], outG[0:npix], outB[0:npix])
292 {
293     #pragma omp teams num_teams(30) default(none)\
294                       shared(inR,inG,inB,outR,outG,outB,radius,width,height,sigma)
295     {
296         #pragma omp distribute parallel for num_threads(128) collapse(2)
```

Όταν χωρίζουμε τη μεταφορά δεδομένων από τον υπολογισμό τότε έχουμε δύο back-to-back offload launches: ένα για το target data και ένα για το target teams. Κάθε launch

εισάγει σημεία συγχρονισμού, με αποτέλεσμα διπλό overhead (μεταφορά + compute) και αύξηση του χρόνου από ~3.15 s σε ~3.49 s.

```
289     #pragma omp target data \
290         map(to:  inR[0:npix], inG[0:npix], inB[0:npix], \
291             radius, width, height, sigma) \
292         map(from: outR[0:npix], outG[0:npix], outB[0:npix])
293     {
294         #pragma omp target teams distribute parallel for collapse(2) \
295             num_teams(30) num_threads(128) \
296             default(none) \
297             shared(inR,inG,inB,outR,outG,outB,radius,width,height,sigma)
```

Συγκεντρώνοντας όλα τα clauses (map, num_teams, num_threads, collapse κ.λπ.) σε ένα μόνο #pragma, ο compiler και το runtime πρέπει να “διαβάσουν” και να ρυθμίσουν όλα τα clauses ταυτόχρονα πριν από το kernel launch. Αυτή η πρόσθετη πολυπλοκότητα στο parsing ανεβάζει ελαφρώς τον χρόνο σε ~3.54 s, σε σύγκριση με πιο “εστιασμένα” pragmas.

```
289     #pragma omp target teams distribute parallel for collapse(2) \
290         num_teams(30) num_threads(128) \
291         map(to:  inR[0:npix], inG[0:npix], inB[0:npix], \
292             radius, width, height, sigma) \
293         map(from: outR[0:npix], outG[0:npix], outB[0:npix]) \
294         default(none) \
295         shared(inR,inG,inB,outR,outG,outB,radius,width,height,sigma)
```

Τέλος, εάν το parallel for τοποθετηθεί έξω από το target, ο βρόχος τρέχει στην CPU, όπου 128 threads σε λίγους φυσικούς πυρήνες προκαλούν συνεχές context-switching και δεν μεταφέρεται τίποτα στη GPU, με αποτέλεσμα ο χρόνος εκτέλεσης να ξεπερνάει αυτόν του σειριακού (~95 s).

```
289     #pragma omp target teams num_teams(30) \
290         map(to:  inR[0:npix], inG[0:npix], inB[0:npix], \
291             radius, width, height, sigma) \
292         map(from: outR[0:npix], outG[0:npix], outB[0:npix]) \
293         default(none) \
294         shared(inR,inG,inB,outR,outG,outB,radius,width,height,sigma)
295     #pragma omp parallel for num_threads(128) collapse(2)
```

Συμπερασματικά, η πιο αποδοτική λύση είναι:

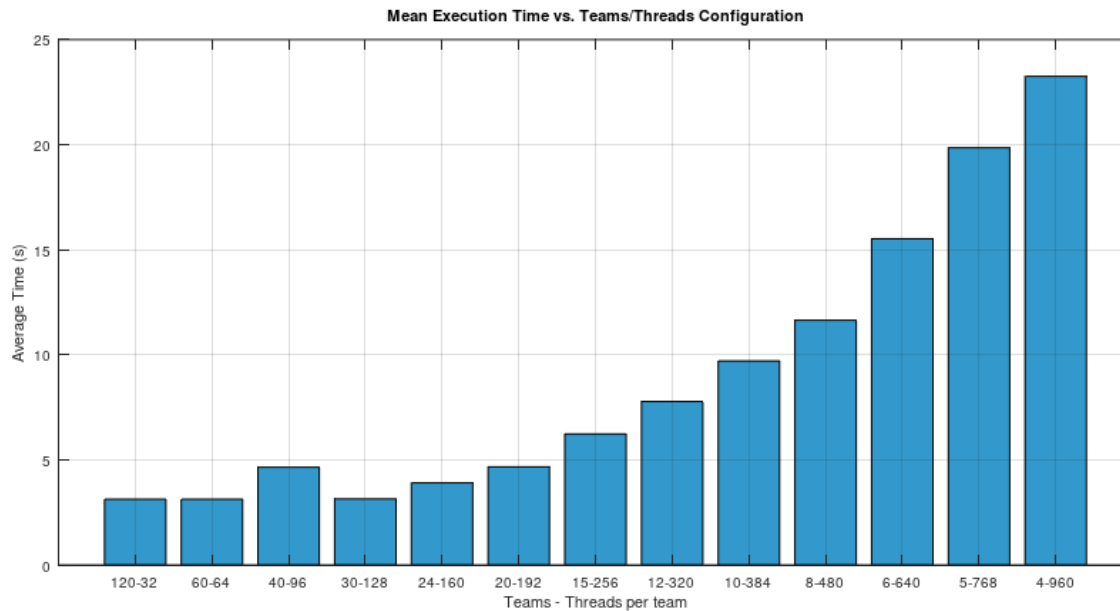
```
289     #pragma omp target map(to:  inR[0:npix], inG[0:npix], inB[0:npix], \
290                             radius, width, height, sigma)\
291                             map(from: outR[0:npix], outG[0:npix], outB[0:npix])
292     {
293         #pragma omp teams num_teams(30) default(none)\
294             shared(inR,inG,inB,outR,outG,outB,radius,width,height,sigma)
295         {
296             #pragma omp distribute parallel for num_threads(128) collapse(2)
```

Πειραματικά αποτελέσματα – μετρήσεις

Δοκιμάζω διάφορους συνδυασμούς ομάδων και νημάτων ανά ομάδα – πάντα μέσα στους περιορισμούς της εκφώνησης – μεταβάλλοντας απλά τις τιμές num_teams(x) (για τον αριθμό των ομάδων) και num_threads(y) (για τον αριθμό των νημάτων ανά ομάδα).

Τα αποτελέσματα είναι από την θόλωση της μεγαλύτερης εικόνας και για ακτίνα θόλωσης ίση με 8.

Πλήθη ομάδων - νημάτων ανά ομάδα	1 ^η Εκτέλεση	2 ^η Εκτέλεση	3 ^η Εκτέλεση	4 ^η Εκτέλεση	Μέσος Όρος
120 – 32	3.152787	3.135172	3.155178	3.111405	3.1386
60 – 64	3.139699	3.108753	3.155384	3.150407	3.1386
40 – 96	4.659776	4.649887	4.650417	4.673428	4.6584
30 – 128	3.151853	3.171455	3.163568	3.123917	3.1527
24 - 160	3.938654	3.908211	3.929584	3.888172	3.9162
20-192	4.699216	4.663406	4.683060	4.666750	4.6781
15-256	6.257229	6.241876	6.263390	6.212595	6.2438
12-320	7.820757	7.744693	7.811974	7.767569	7.7863
10-384	9.376533	9.358973	9.361234	10.769974	9.7167
8-480	11.616372	11.657028	11.667617	11.662332	11.6508
6-640	15.527938	15.535134	15.530359	15.522742	15.5290
5-768	22.244305	19.943016	18.622873	18.613399	19.8559
4-960	23.255591	23.241416	23.263119	23.257374	23.2544



Σχόλια

Οι κορυφαίοι χρόνοι (3.13–3.15 s) επιτυγχάνονται όταν

$$\#teams \times \#threads = 3840 = 30SMs \times 128threads/SM.$$

Παρατηρούμε ότι στους παρακάτω συνδυασμούς καλύπτονται ακριβώς όλοι οι πυρήνες της GPU χωρίς αδράνεια ή υπερφόρτωση, και έτσι ελαχιστοποιείται το idle time.

- 120 teams × 32 threads = 3840
- 60 teams × 64 threads = 3840
- 30 teams × 128 threads = 3840

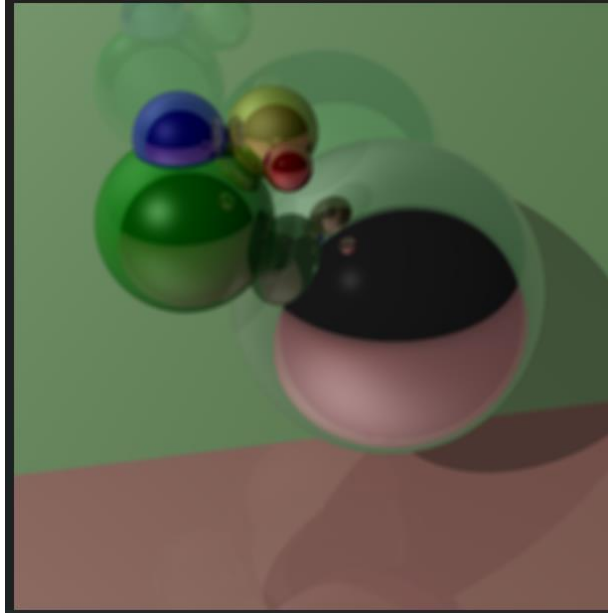
Όταν δώσουμε π.χ. 40 teams × 96 threads = 3840 συνολικά, έχουμε 40 ομάδες που πρέπει να εκτελεστούν σε μόλις 30 SMs. Έτσι κάθε SM αναγκάζεται να χειριστεί περισσότερες από μία ομάδες, με αποτέλεσμα παραπάνω context-switching μέσα στη GPU και πτώση απόδοσης (4.66 s).

Για 24 teams × 160 threads, πάλι 3840 συνολικά, αλλά μόλις 24 ομάδες, 6 SMs μένουν άδεια, οπότε 6 SMs δουλεύουν διπλό load και σπαταλάει threads σε software scheduling, ανεβάζοντας το χρόνο σε ~3.92 s.

Όταν περνάμε σε πολύ λίγες ομάδες (π.χ. 4 teams × 960 threads), ελάχιστα SMs αναλαμβάνουν τεράστιο load και όλα τα υπόλοιπα είναι idle — συνεπώς ο χρόνος σκαρφαλώνει στα 23 s, σχεδόν σαν να μην είχαμε καθόλου offload.

Συμπερασματικά, οι καλύτερες επιδόσεις επιτυγχάνονται όταν ακριβώς η #teams, #threads ισούται με 30 SMs, 128 threads/SM. Κάθε απόκλιση (περισσότερες ομάδες, λιγότερα threads ανά ομάδα ή αντίστροφα) δημιουργεί είτε αδράνεια (idle SMs) είτε υπερφόρτωση (υπο-ή υπερβολικό scheduling), κι έτσι αυξάνει τον χρόνο εκτέλεσης.

Παρακάτω φαίνεται η θολωμένη εικόνα:



Πώς αλλάζεις T και N:

- Νήματα ανά ομάδα = `num_threads(N)` → πρέπει $N \% 32 == 0$
- Ομάδες = `num_teams(T)` → πρέπει $T \times N == 3840$

Παράδειγμα:

- $N=32 \rightarrow T=3840/32=120$
- $N=64 \rightarrow T=3840/64=60$
- $N=128 \rightarrow T=3840/128=30$
- $N=256 \rightarrow T=3840/256=15$